

**INSTITUTO FEDERAL DE  
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**  
**SUL-RIO-GRANDENSE**  
**Campus Passo Fundo**

**Bacharelado em Ciência da Computação**  
**Arquitetura de Computadores I**

## **Conteúdo 4:**

### **Processador:**

### **Unidade Central de Processamento**

**Prof. Petry, Carlos A**



Baseado no livro:  
Organização e Projeto de Computadores  
David Patterson

# INTRODUÇÃO

- Processadores possuem dois caminhos (path) principais

- Caminho de Dados (Data Path)

Representa o caminho por onde os dados movimentam-se para/do Processador

- Caminho de Controle (Control Path)

Representa o caminho por onde são enviados sinais de controle e sincronização de HW

- Processadores são construídos por 4 componentes fundamentais:

- Lógica Combinacional: elementos de cálculo
  - Lógica Sequencial: elementos de armazenamento
  - Caminho de dados: fluxo de dados do processamento
  - Caminho de controle: fluxo dos sinais de orquestração do processamento

# INTRODUÇÃO: IMPLEMENTAÇÃO BÁSICA DO PROCESSADOR MIPS

- Subconjunto básico do conjunto de instruções
  1. Instruções de referência à **memória**: **load word** (lw) e **store word** (sw)
  2. Instruções **logico-aritmética**: add, sub, AND, OR e slt
  3. Instruções de **desvio**: **branch equal** (beq) e **jump** (j)
- Instruções não consideradas
  - Instruções com inteiros: shift, multiply, divide
  - Instruções para Float Point

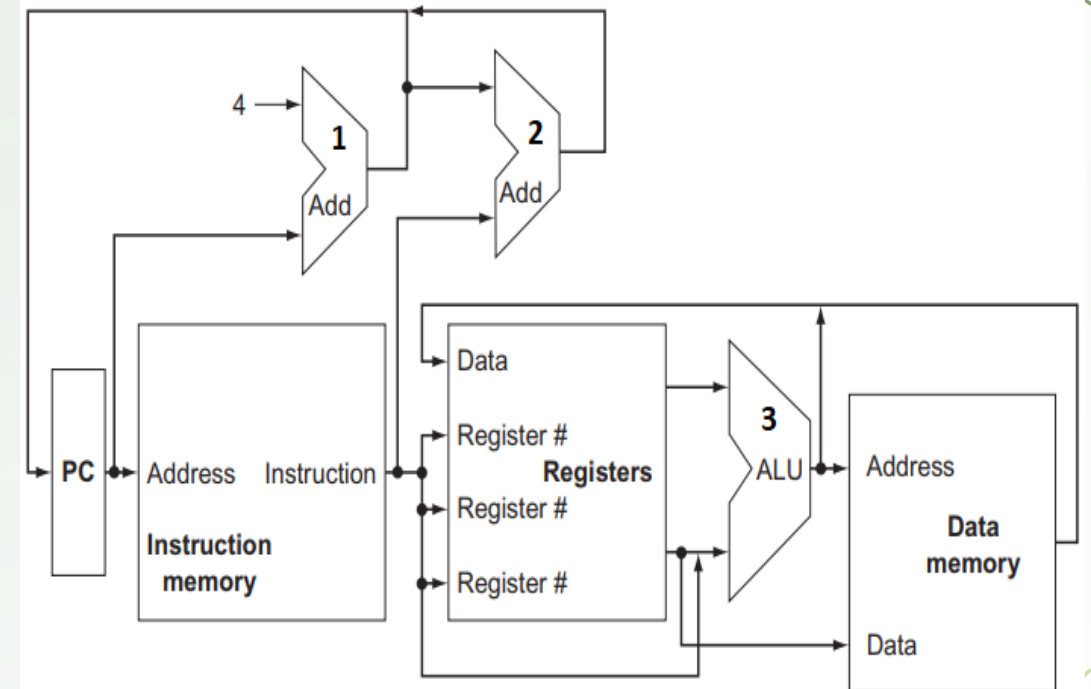
# INTRODUÇÃO: RESUMO DA IMPLEMENTAÇÃO

- Etapas comuns ao processamento de instruções
  1. **Busca da instrução** em memória conforme endereço no **registrador PC**
  2. **Decodificação**, analisa a instrução recebida, decodifica e gera os sinais de controle
  3. **Execução**: inicia com a **leitura de registradores**, selecionados **da instrução** buscada/decodificada
- Os **passos seguintes** da execução **dependem da classes** da instrução:
  - Acesso à memória
    - ALU→ cálculo do endereço
    - Memória→ leitura ou escrita
  - Lógico-aritmética
    - ALU→ cálculos
    - Registrador→ escrita do resultado
  - Desvio
    - ALU→ comparação (exceto j)
    - PC→ somar 4 ou o endereço da próxima instrução (se tomado)

# INTRODUÇÃO: DIAGRAMA DA IMPLEMENTAÇÃO MIPS

- Diagrama alto nível simplificado das unidades funcionais e interconexões

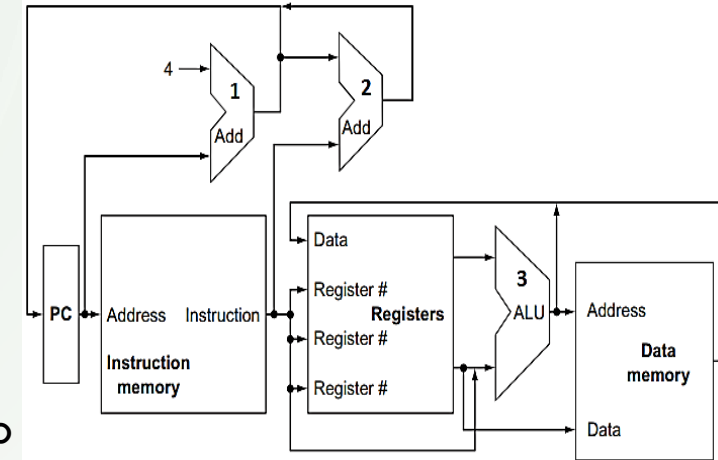
- Memória Instrução:** armazena as instruções a serem executadas
- Memória Dados:** armazena os dados manipulados pelas instruções
- PC:** endereça a próxima instrução a buscar na memória de instrução
- Registradores:** armazena os dados manipulados dentro do processador
- ALU 1 e 2:** usadas no cálculo da instrução seguinte, (1) próxima, (2) em desvio tomado
- ALU 3:** usada nos cálculos logico-aritméticos e comparação (desvio)



# INTRODUÇÃO: DIAGRAMA DA IMPLEMENTAÇÃO MIPS

- Funcionamento simplificado das unidades funcionais e interconexões

- Instrução é buscada da memória de instrução no endereço  $\rightarrow$  PC
- Operandos obtidos conforme campos da instrução ( $R_s, R_d, R_t, Imm$ )
- ALUs realiza cálculos:
  - Load e Store  $\rightarrow$  endereços de memória
  - Logico-aritmético  $\rightarrow$  operações matemáticas
  - Desvio  $\rightarrow$  operações de comparação, visando tomar ou não o desvio  $\rightarrow$  definir endereço da instrução seguinte



- Resultados

- Load e Store  $\rightarrow$  leitura/escrita para/de registrador em memória
- Logico-aritmético  $\rightarrow$  escrita em registrador
- Desvio  $\rightarrow$  PC recebe o endereço da instrução seguinte

- Notação:

Linhas  $\rightarrow$  barramentos (Bus), Setas  $\rightarrow$  direção do fluxo de bits, Ponto ( $\bullet$ )  $\rightarrow$  interconexão de Bus

# INTRODUÇÃO: DIAGRAMA CAMINHO DE DADOS E CONTROLE

- Diagrama simplificado contendo o caminho de dados e controle

- Mux**, 3 multiplexadores para **seleção** do:

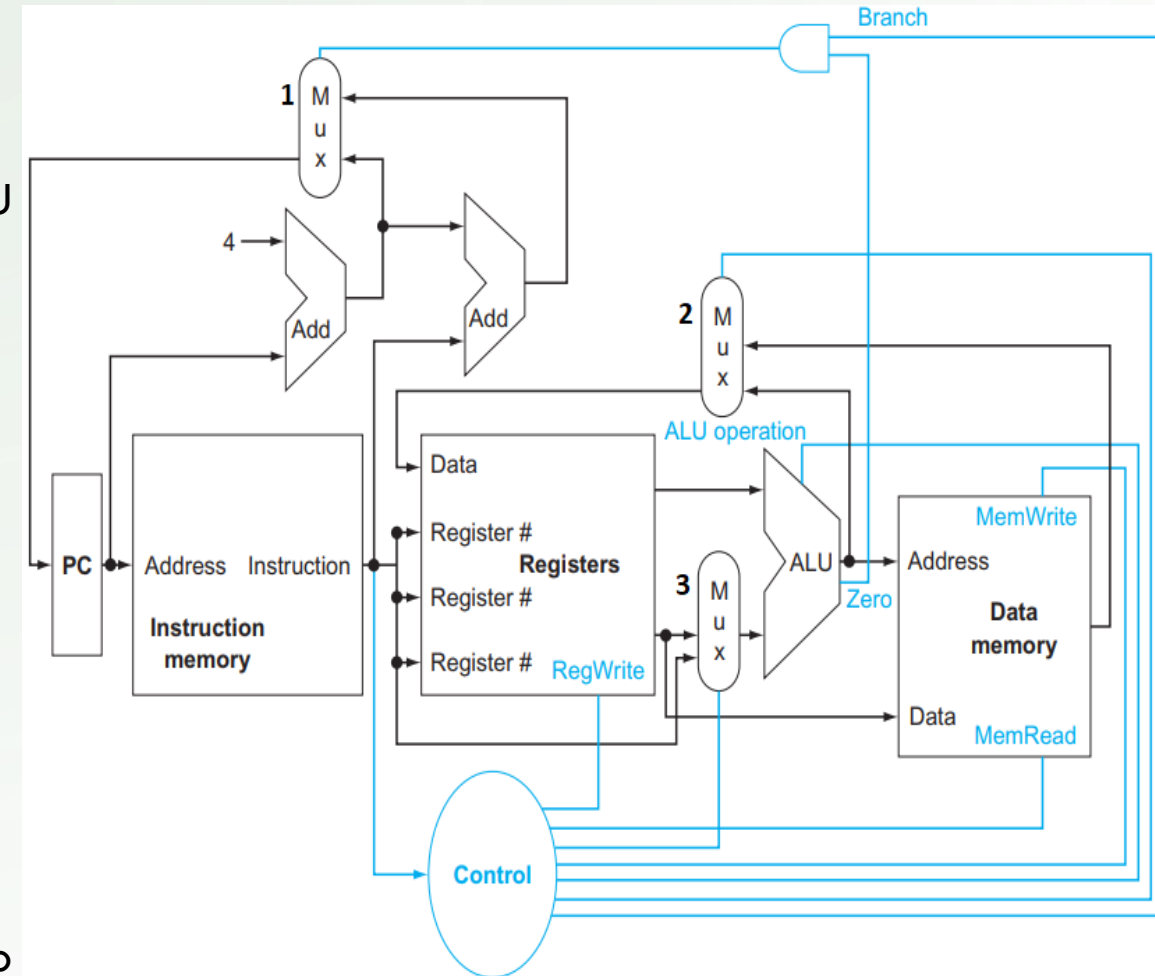
- Endereço** sequencial ou de desvio do PC
- Dado ao banco registrador** da memória ou ALU
- Dado para ALU** da instrução ou registrador

- Linhas de controle:**

- Diversos sinais de controle
- Enviados para as várias unidades funcionais
- Coordena a execução das instruções

- Unidade de controle**

- Recebe a instrução da memória
- Decodifica a instrução
- Gera e emite os sinais de controle/sincronização





# CONVENÇÕES LÓGICAS DO PROJETO

- Convenções lógicas definem
  - Como as unidades funcionais devem operar
  - Como as unidades funcionais serão sincronizadas
- Unidades funcionais são compostos por dois tipos de elementos lógicos
  - Combinacional
  - Sequenciais



# CONVENÇÕES LÓGICAS DO PROJETO

- Elemento **combinacional**

- Elementos que **realizam operações**, exemplo: ALU, Mux, portas lógicas
- Saída: (1) depende apenas dos valores da entrada (2) são “imediatamente” calculadas a partir das entradas

- Elementos **sequenciais** ou de estado

- Elementos que **armazenam dados**, exemplo: registradores, memórias, Flip-flop
- Saída: corresponde aos valores internamente armazenados (previamente)
- Entradas: compreendem (1) ao valor a armazenar e (2) sinal de clock

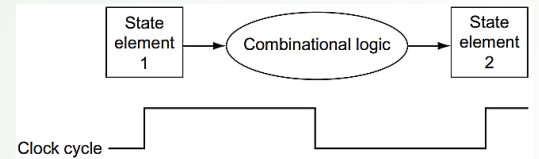
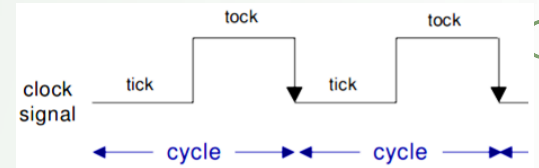
- Metodologia de **clock**

- Determina quando e como os **sinais** devem ser **gerados e interpretados**
- Determina quando os **dados** podem ser **lidos** ou **escritos** (estão disponíveis/estáveis)

# CONVENÇÕES LÓGICAS DO PROJETO

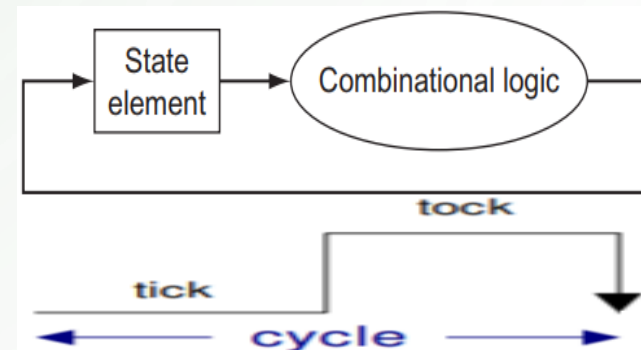
- Sincronização acionada por transição

- Um ciclo de clock possui duas transição: de subida e de descida
- Elementos sequenciais
  - Valores armazenados são atualizados apenas na ocorrência de transição
- Elementos combinacionais
  - Não armazenam valores , apenas calculam
  - Portanto: recebe e enviam dados de/para elementos sequencias
- Sistema digitais síncronos e elementos sequenciais/combinacionais
  - Estão interrelacionados e são sincronizados pelo clock (transição)
  - Estradas escritas podem ser lidas no ciclo seguinte
  - Sinal de clock: propaga-se do elemento 1 até o 2, passando pela lógica combinacional



# CONVENÇÕES LÓGICAS DO PROJETO

- **Sincronização** acionada por transição no **MIPS**
  - Os elementos lógicos são acionados por **transição positiva**
  - Portanto, são **sincronizados** na transição de **subida do clock**
  - Sinais de **controle** e **clock** representam **entradas** em elementos lógicos
  - Elementos sequenciais são alterados quando:  
o sinal de **controle** de escrita está **ativo** e ocorrer a **transição positiva** do clock
- A **sincronização** por **transição** permite:
  1. Ler dados de um registrador
  2. Executar a operação no elemento combinacional
  3. Escrever o resultado em um registrador
  - As 3 **etapas** acima podem ser **realizadas** em **um** único **ciclo** de clock



# CONVENÇÕES LÓGICAS DO PROJETO

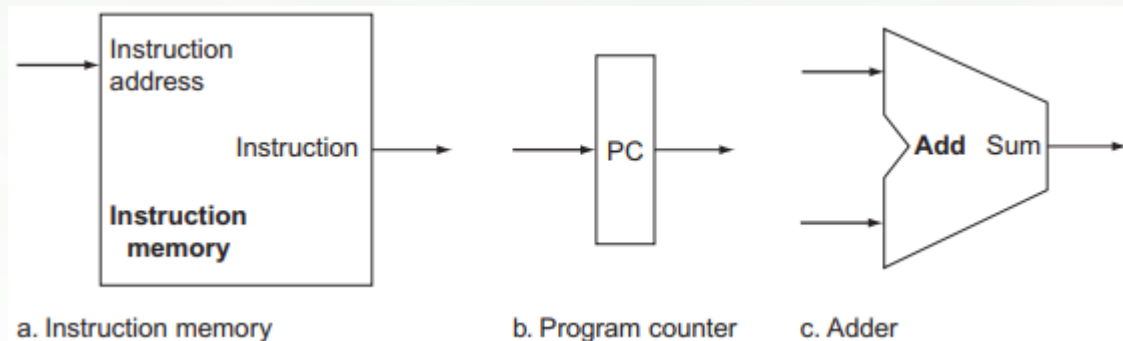
- Convenções para termos
  - **Ativo/Inativo**: indica se um sinal (controle/clock) esta **Alto** ou **Baixo**, respectivamente
  - **Ativar**: especifica que um sinal deve ser **conduzido** para o nível **Alto**
  - **Desativar**: especifica que um sinal deve ser **conduzido** para o nível **Baixo**
- Convenções do projeto
  - Todos os **elementos** sequenciais e combinacionais **possuem** tamanho de **32 bits** ou **1 bit**, **exceções** ( $\neq 32b$ ) serão explicitamente **indicadas** via rótulo
  - **Linhas** representam **barramentos**: controle, endereço ou dados
  - **Setas**: indicam a **direção** que os **sinais** seguem (fluxo: dados ou controle)

# O CAMINHO DE DADOS: DATAPATH

- Projeto/estudo baseado em **instruções** como **exemplo** de caso
- Determinar
  - Elementos lógicos necessários
  - Sinais de controle obrigatórios
  - Fluxo dos sinais entre os elementos lógicos
- Elementos do caminho de dados
  - **Unidades funcionais** necessárias para **operar** ou **armazenar** dados
  - Memórias: instruções e dados
  - Registradores: PC e banco
  - Unidade lógico-aritmética (ALU) e somadores

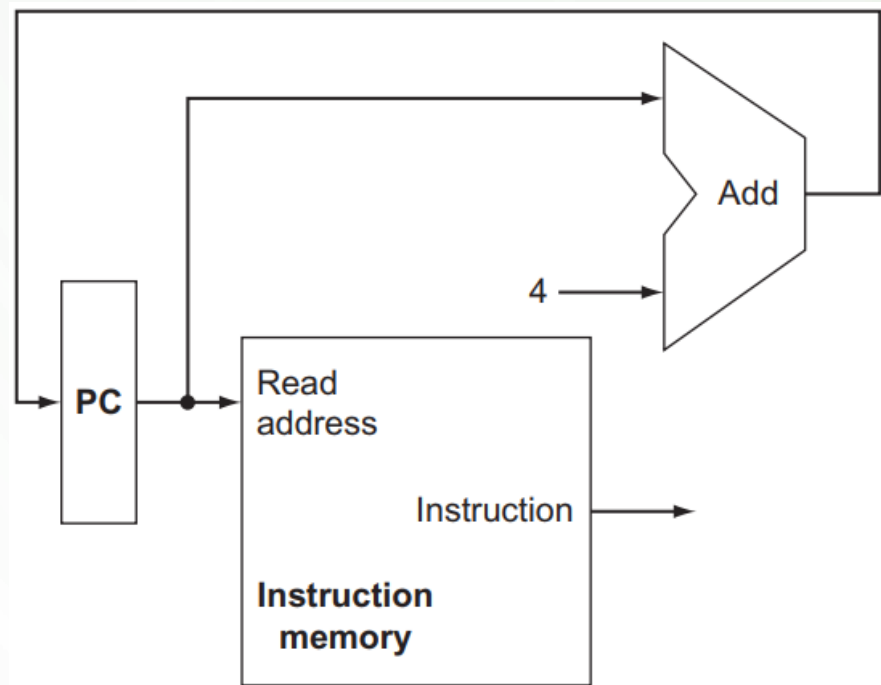
# O CAMINHO DE DADOS: ELEMENTOS

- Memória de instruções (estado)
  - Armazena instruções de um programa
  - Disponibiliza acesso às instruções: leitura
- Registradores (estado)
  - Elementos de armazenamento interno: exemplo PC (32b), endereço da instrução em execução
- Somador (cálculo)
  - Elemento combinacional que realiza somas: exemplo, obter o endereço da próxima instrução



# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de uma instrução
  - Iniciamos **buscando** a **instrução** na memória de instruções
  - **Incrementamos** o **PC** (1 word ou 4B) para indicar a próxima instrução
- Implementamos estes passos com o seguinte Datapath





# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de instruções do formato R

- Registradores de uso geral: inseridos no Banco de Registrados (BR)
  - Endereçados via números 0 a 31, tamanho 32 bits, podem ser lidos ou escritos
  - Precisamos 3 entradas no BR para selecionar registradores
  - Escritas são controladas pelo sinal `RegWrite`
  - Leituras são realizadas apenas endereçando o registrador

- Formato

| Nome             | Campos |        |        |        |        |        |
|------------------|--------|--------|--------|--------|--------|--------|
| Tamanho do campo | 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |
| Formato R        | op     | rs     | rt     | rd     | shamt  | funct  |

- Instruções R (`add`, `sub`, `AND`, `OR` e `slt`) usam 3 registradores
- Leem 2 registradores, realizam operações na ALU, escrevem o resultado em um registrador

# O CAMINHO DE DADOS: EXECUÇÃO

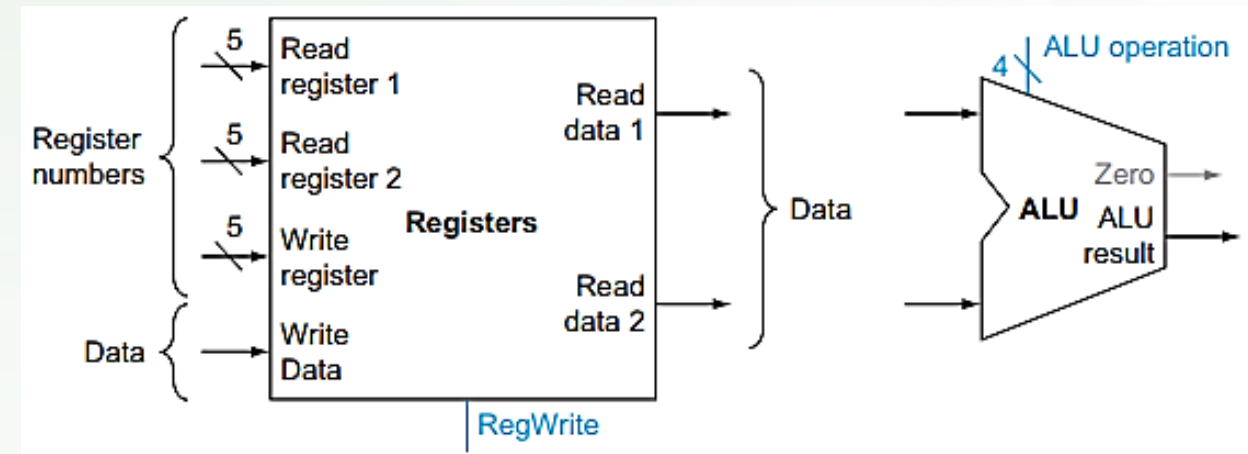
- Execução de instruções do formato R

- Caminhos do BR:

- Endereço:
  - 2 caminhos para leitura 5b: ReadRegister1 e ReadRegister2
  - 1 caminhos para escrita 5b: WriteRegister
- Dados
  - 2 caminhos para saída 32b : ReadData1 e ReadData2
  - 1 caminhos para entrada 32b: WriteData
- Controle
  - 1 sinal de habilitação de escrita 1b: RegWrite

- Caminhos da ALU

- Dados
  - 2 caminhos para saída 32/1b: Result e Zero
  - 2 caminhos para entrada 32b: Data1 e Data2
- Controle
  - 1 sinal 4b código operação: ALU Operation



# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de instruções do **formato I**

- Formato

| Nome             | Campos |        |        |                   |        |        |
|------------------|--------|--------|--------|-------------------|--------|--------|
| Tamanho do campo | 6 bits | 5 bits | 5 bits | 5 bits            | 5 bits | 6 bits |
| Formato I        | op     | rs     | rt     | endereço/imediato |        |        |

- **Instruções I** (`lw` e `sw`) usam 2 registradores e um imediato de 16 bits com sinal

- **Exemplo:** `lw $t1, offset($t2)` ou `sw $t1, offset($t2)`

- **Load:**

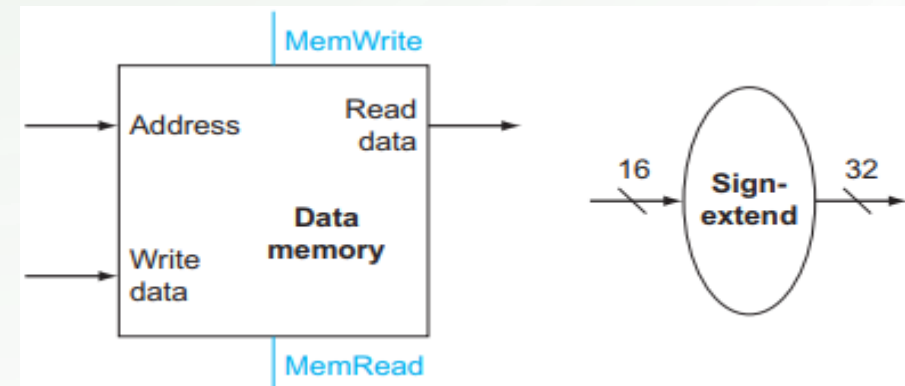
- Endereça a memória pela extensão de sinal de 16b para 32b somado ao 2º registrador
    - Armazena o valor vindo da memória no 1º registrador

- **Store:**

- Endereça a memória pela extensão de sinal de 16b para 32b somado ao 2º registrador
    - Armazena na memória o valor vindo no 1º registrador

# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de instruções do formato I: load/store
  - Precisamos de um novo elemento lógico: “**extensão de sinal**” usando o imediato de 16b
  - O imediato de 16 bits é estendido para 32b para ser somando ao 2º registrador
  - A **memória de dados** pode ser **lida** ou **escrita**: precisamos de **2 sinais de controle**
- Caminhos da Memória de dados:
  - Endereço:
    - 1 caminhos para acesso 32b: Address
  - Dados
    - 1 caminhos para saída 32b : ReadData
    - 1 caminhos para entrada 32b: WriteData
  - Controle
    - 2 sinal de habilitação 1b: MemWrite, MemRead
- Caminhos da extensão de sinal
  - Entrada
    - 1 caminhos de entrada 16b: DataIn
  - Saída
    - 1 caminhos de saída 32b sign-extended: DataOut



# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de instruções do **formato I**

- Formato

| Nome             | Campos |        |        |                   |        |        |
|------------------|--------|--------|--------|-------------------|--------|--------|
| Tamanho do campo | 6 bits | 5 bits | 5 bits | 5 bits            | 5 bits | 6 bits |
| Formato I        | op     | rs     | rt     | endereço/imediato |        |        |

- **Instruções I** (beq) usa 2 registradores e um imediato (constante) de 16 bits com sinal

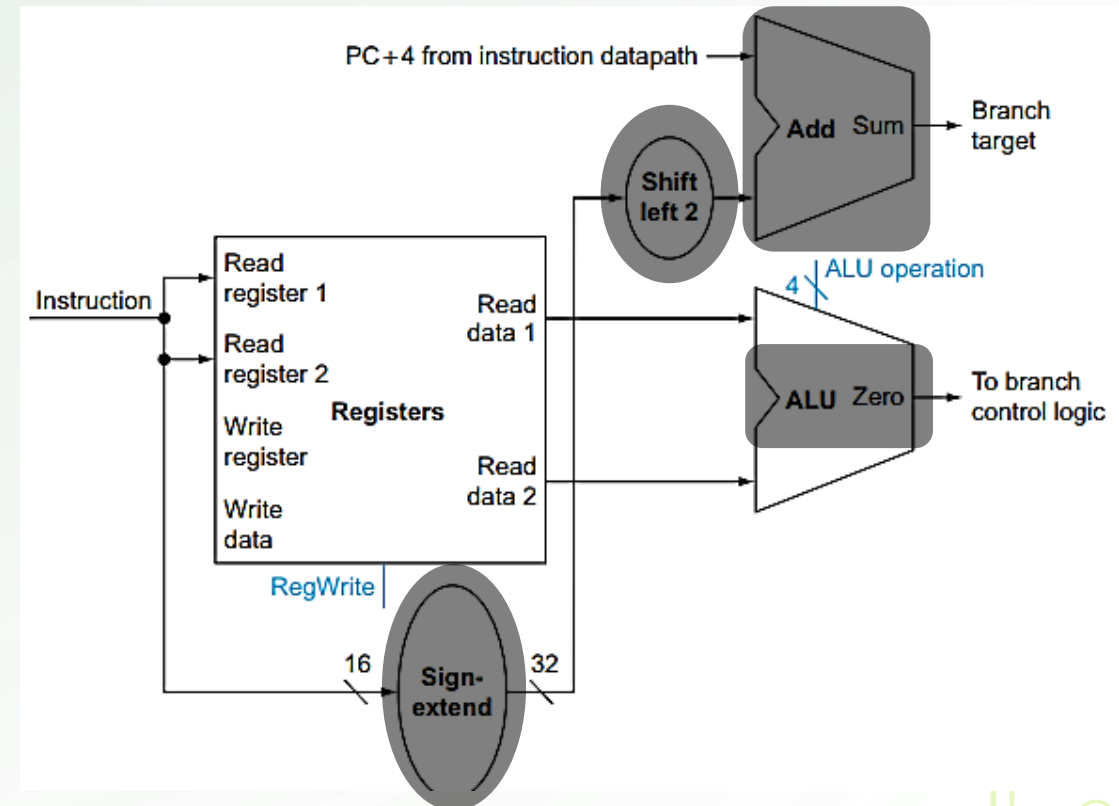
- Exemplo: beq \$t1, \$t2, offset

- **Cálculo do endereço de deslocamento** (offset)

- O deslocamento deve considerar a instrução seguinte à instrução branch
    - Assim, somamos o imediato de 16b estendido para 32b com o PC + 4
    - PC+4 já é calculado no fluxo projetado (veja slide 15)
    - O **offset** é **deslocado em 2 bits** à esquerda, ou seja, endereça palavras (**word**) não bytes
    - **beq** compara os 2 registradores:
      - sendo iguais (true) o desvio é **tomando** → segue o fluxo desviado
      - sendo diferentes (false) o desvio **não** é **tomando** → segue o fluxo sequencial

# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de instruções do formato I: branch
  - Instruções branch precisam realizar 2 operações:
    - Calcular o endereço de destino do desvio: offset
    - Comparar o valor dos 2 registradores: true/false
- Caminhos para instruções branch:
  - Extensão de sinal:
    - 2 caminhos entrada/saída 16b/32b
  - Deslocamento do offset
    - 2 caminhos entrada/saída 32b/32b: deslocamento 2 bits à esquerda
  - Somador
    - 2 caminhos para entrada 32b: offset e PC+4
    - 1 caminhos para saída 32b: endereço BranchTarget
  - ALU
    - 1 caminhos para saída 1b: resultado comparação 0=true 1=false



# O CAMINHO DE DADOS: EXECUÇÃO

- Execução de instruções do **formato J**

- Formato

| Nome             | Campos |                     |        |        |        |        |
|------------------|--------|---------------------|--------|--------|--------|--------|
| Tamanho do campo | 6 bits | 5 bits              | 5 bits | 5 bits | 5 bits | 6 bits |
| Formato J        | op     | endereço de destino |        |        |        |        |

- **Instruções J** (j) não usa registradores, apenas um imediato de 26 bits com sinal
  - Exemplo: j offset
  - **Cálculo do endereço alvo** (target)
    - O **offset** é **deslocado em 2 bits** à esquerda, endereça palavras, gerando endereço de 28 bits
    - Concatenamos os 4 bits superiores do PC ao offset de 28b para gerar o endereço alvo de 32b
    - A instrução seguinte ao desvio (PC+4) é executada antes do desvio
    - **j** não faz nenhuma comparação, apenas devia para o endereço calculado
    - O elemento lógico para jump é simples
$$\text{Upper\_4b(PC)} + \text{Offset\_26b} + 00 \rightarrow \text{TargetAddress}$$
(+ indica concatenação)



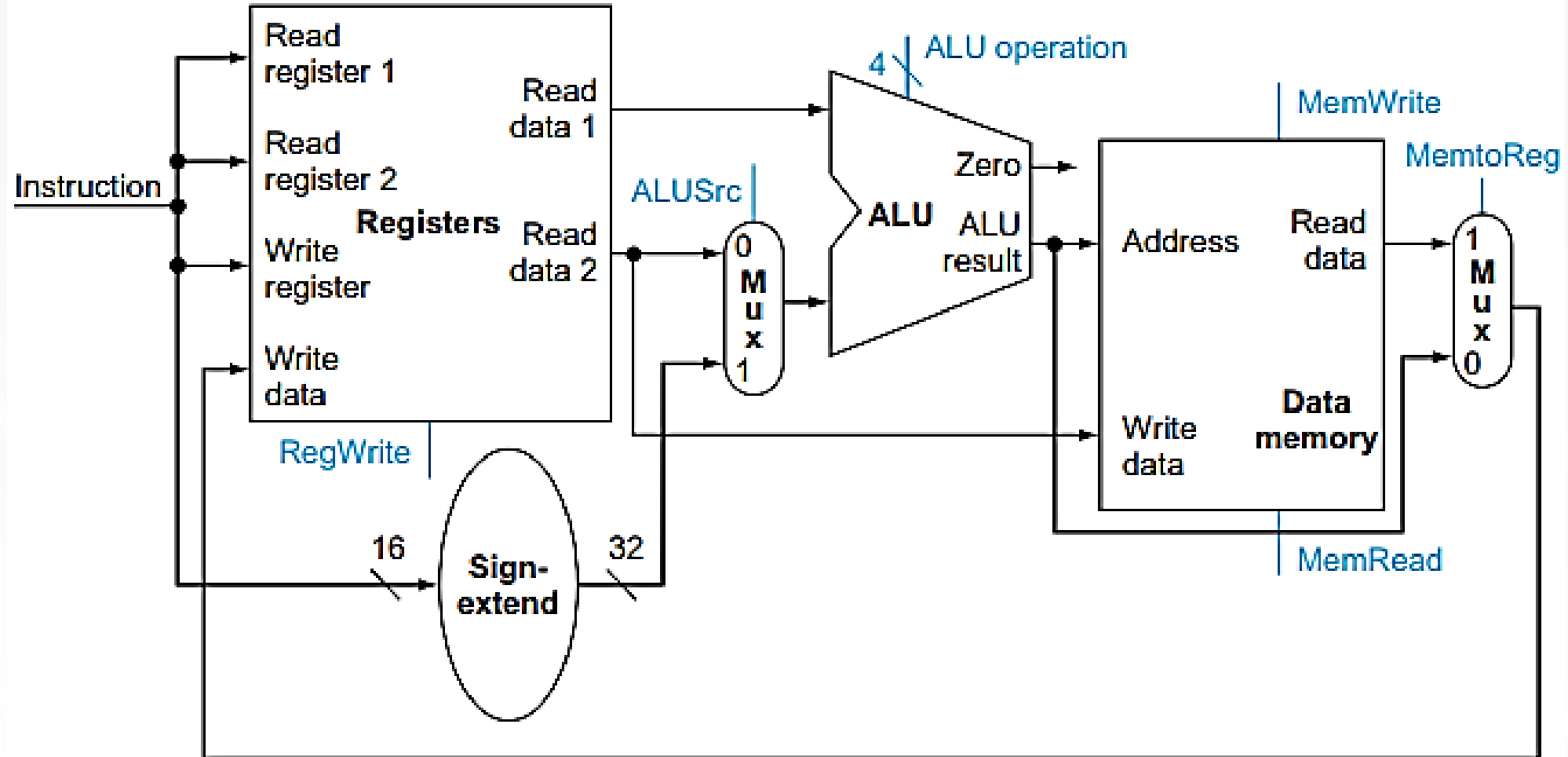
# O CAMINHO DE DADOS: COMBINADOS

- Analisamos os caminhos para as instruções de formato R, I e J
- Combinando os caminhos, podemos gerar um diagrama lógico que executa as instruções R, I e J adicionando sinais de controle e multiplexadores (Mux)
- Precisamos **duas origens** para registradores do BR:
  - a **saída** de dados do **2º registrador** do BR → registrador ou imediato
  - a **entrada** de dados do **BR** → saída da **ALU** ou leitura da **memória** de dados



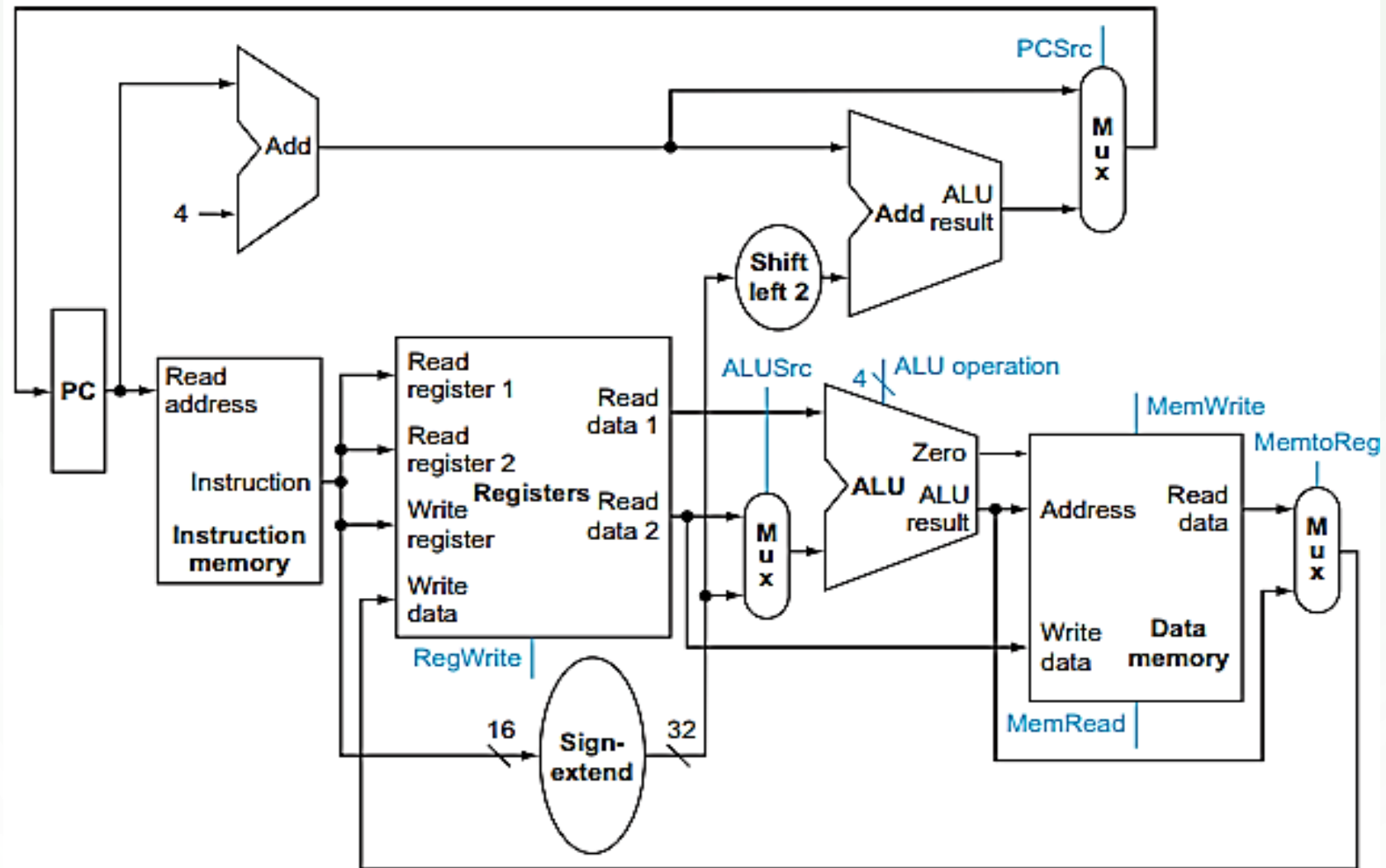
# O CAMINHO DE DADOS: COMBINADOS

- Caminhos combinados compartilhando entrada e saída do Banco de Registradores (BR)



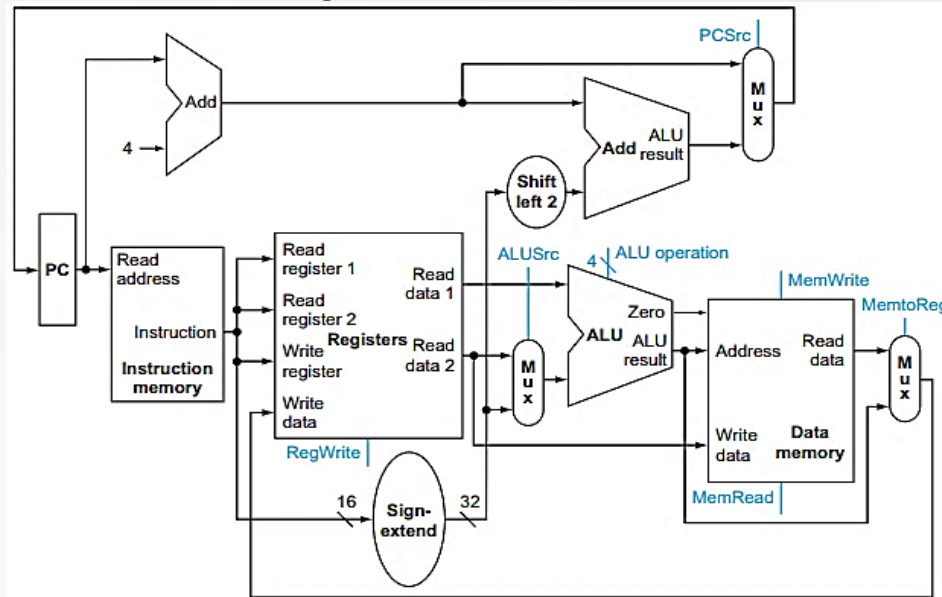
# O CAMINHO DE DADOS: COMBINADOS

- Caminhos combinados incluído todos os elementos lógicos e sinais de controle



# O CAMINHO DE DADOS: IMPLEMENTAÇÃO SIMPLES

- Possível implementação do caminhos de dados combinado

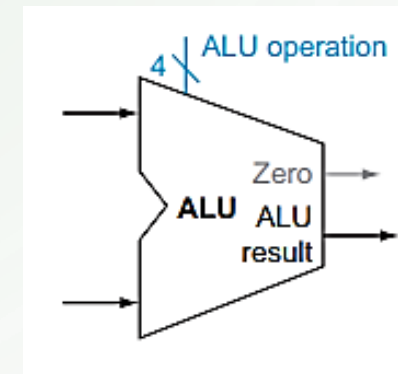


- A implementação deve suportar a execução de instruções:
  - Entrada e Saída: `load`, `store` (`lw`, `sw`)
  - Desvio condicional: `branch equal` (`beq`)
  - Logico-aritmética: `add`, `sub`, `AND`, `OR`, `slt` (`nor`)
  - Desvio incondicional: `jump` (`j`)

# O CAMINHO DE DADOS: IMPLEMENTAÇÃO SIMPLES

- A unidade logica e aritmética (ALU) realiza várias operações: 6 no total
- Precisamos indicar que operação desejamos da ALU
- Precisamos do sinal de controle **ALU Operation** para selecionar as operações
- Usamos **4 bits** para emitir o sinal de controle para a **ALU Operation**

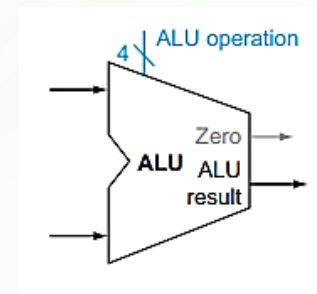
| ALU control lines | Function         |
|-------------------|------------------|
| 0000              | AND              |
| 0001              | OR               |
| 0010              | add              |
| 0110              | subtract         |
| 0111              | set on less than |
| 1100              | NOR              |



# O CAMINHO DE DADOS: IMPLEMENTAÇÃO SIMPLES

- Cada classe de instrução usa uma das 5 função da ALU
  - Load / Store:
    - Usam `add` para calcular o endereço de memória ( `16(r1)` )
  - Instruções tipo R:
    - Usam 5 operações: `AND`, `OR`, `add`, `sub`, `slt` , `nor`
    - Selecionadas através do campo `funct` da instrução (6 bits finais)
- Uma unidade de controle pode gerar os 4 bits da ALU, combinando:
  - Entrada `funct`: 6 bits menos significativos da instrução
  - Entrada `ALUOp`: 2 bits indicando a operação a realizar
    - 00 → `add`: para load e store (`lw`, `sw`)
    - 01 → `subtract`: para desvio (`beq`)
    - 10 → operação definida pelo campo `funct` da instrução
- Saída operação da ALU
  - xxxx → 4 bits que definem a operação a realizar na ALU

| ALU control lines | Function         |
|-------------------|------------------|
| 0000              | AND              |
| 0001              | OR               |
| 0010              | add              |
| 0110              | subtract         |
| 0111              | set on less than |
| 1100              | NOR              |



# O CAMINHO DE DADOS: IMPLEMENTAÇÃO SIMPLES

- A tabela mostra como as entradas da unidade de controle da **ALU** definem **ALU-control**

| Opcode      | Instruction opcode | ALUOp | Instruction operation | Funct field | Desired ALU action | ALU control input |
|-------------|--------------------|-------|-----------------------|-------------|--------------------|-------------------|
| 23/100011   | LW                 | 00    | load word             | XXXXXX      | add                | 0010              |
| 2B/101011   | SW                 | 00    | store word            | XXXXXX      | add                | 0010              |
| 04/000100   | Branch equal       | 01    | branch equal          | XXXXXX      | subtract           | 0110              |
| 0,20/100000 | R-type             | 10    | add                   | 100000      | add                | 0010              |
| 0,22/100010 | R-type             | 10    | subtract              | 100010      | subtract           | 0110              |
| 0,24/100100 | R-type             | 10    | AND                   | 100100      | AND                | 0000              |
| 0,25/100101 | R-type             | 10    | OR                    | 100101      | OR                 | 0001              |
| 0,2A/101010 | R-type             | 10    | set on less than      | 101010      | set on less than   | 0111              |

- Opcode** → define os valores de **AluOp**
- AluOp** 00 e 01 → não dependem do campo **funct**, valores de ALU-control são definidos pela lógica ( XXXXXX → indica don't care na coluna 'Funct field' )
- AluOp** 10 → valores do campo **funct** determinam os valores de ALU-control

# O CAMINHO DE DADOS: IMPLEMENTAÇÃO SIMPLES

- A tabela verdade da unidade de controle da ALU que define ALU-control

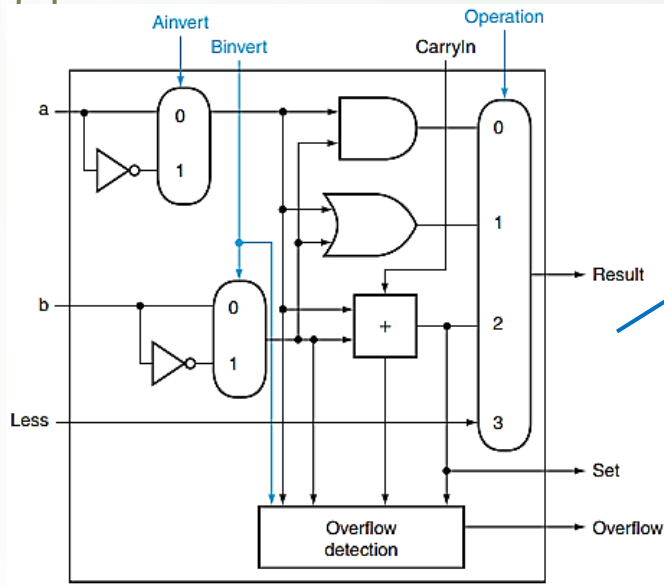
| ALUOp  |        | Funct field |    |    |    |    |    | Operation |
|--------|--------|-------------|----|----|----|----|----|-----------|
| ALUOp1 | ALUOp0 | F5          | F4 | F3 | F2 | F1 | F0 |           |
| 0      | 0      | X           | X  | X  | X  | X  | X  | 0010      |
| X      | 1      | X           | X  | X  | X  | X  | X  | 0110      |
| 1      | X      | X           | X  | 0  | 0  | 0  | 0  | 0010      |
| 1      | X      | X           | X  | 0  | 0  | 1  | 0  | 0110      |
| 1      | X      | X           | X  | 0  | 1  | 0  | 0  | 0000      |
| 1      | X      | X           | X  | 0  | 1  | 0  | 1  | 0001      |
| 1      | X      | X           | X  | 1  | 0  | 1  | 0  | 0111      |

| Instruction opcode | ALUOp | Desired ALU action | ALU control input |
|--------------------|-------|--------------------|-------------------|
| LW                 | 00    | add                | 0010              |
| SW                 | 00    | add                | 0010              |
| Branch equal       | 01    | subtract           | 0110              |
| R-type             | 10    | add                | 0010              |
| R-type             | 10    | subtract           | 0110              |
| R-type             | 10    | AND                | 0000              |
| R-type             | 10    | OR                 | 0001              |
| R-type             | 10    | set on less than   | 0111              |

- Entradas: ALUOp e Funct Field
- Linhas da tabela verdade: apenas valores que ativam a ALU
  - O número de combinações totais seria  $2^8=256$
  - Porém várias são irrelevantes (Saída=0), assim apenas 7 são relevantes para a ALU
  - X...X → indica don't care nas colunas

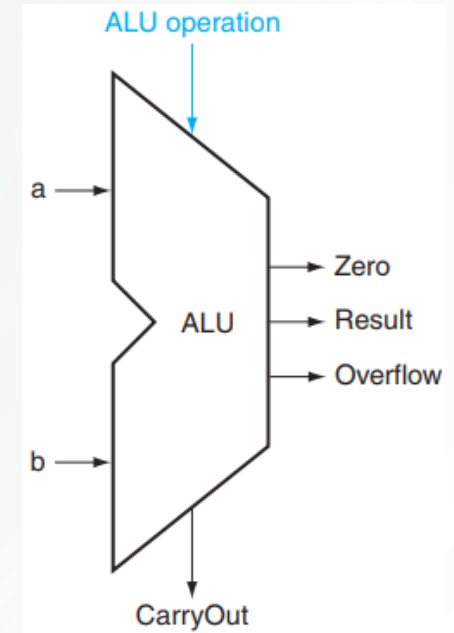
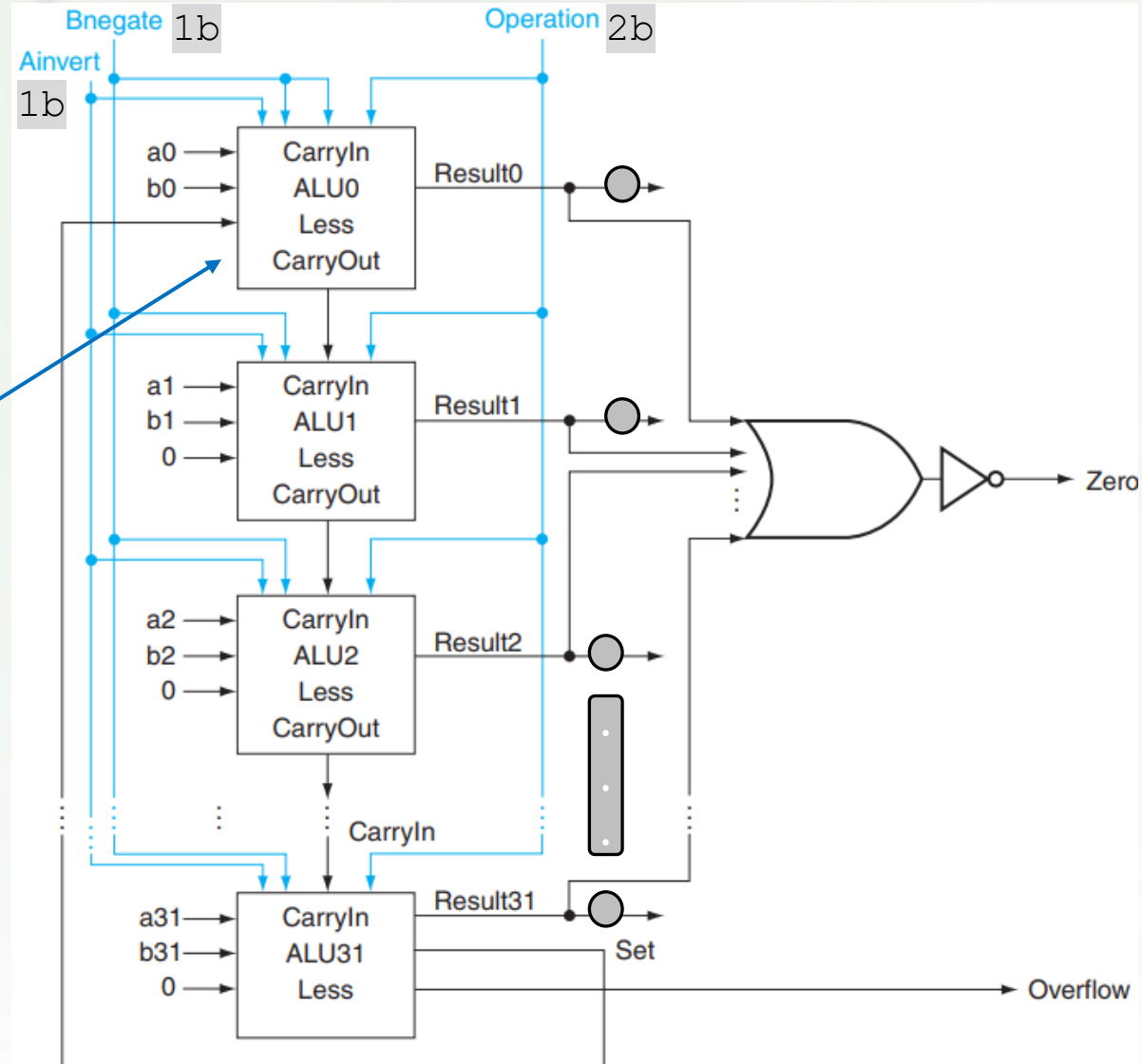


# ALU: IMPLEMENTAÇÃO SIMPLES



4b

| ALU control lines | Function         |
|-------------------|------------------|
| 0000              | AND              |
| 0001              | OR               |
| 0010              | add              |
| 0110              | subtract         |
| 0111              | set on less than |
| 1100              | NOR              |



# ALU: PROJETO

- Definido o controle da ALU (Alu-control), continuamos a **desenvolver o projeto**
- Precisamos conectar os **campos** da instrução ao **caminho de dados**
- Avaliando os **campos das instruções**, temos:
  - a. **R-Type**: opcode 0 + funct
  - b. **Load/Store**: opcodes 35(23) e 43(2B)
  - c. **Branch**: opcode 4 (04)

|                              |          |       |       |         |       |       |
|------------------------------|----------|-------|-------|---------|-------|-------|
| Field                        | 0        | rs    | rt    | rd      | shamt | funct |
| Bit positions                | 31:26    | 25:21 | 20:16 | 15:11   | 10:6  | 5:0   |
| a. R-type instruction        |          |       |       |         |       |       |
| Field                        | 35 or 43 | rs    | rt    | address |       |       |
| Bit positions                | 31:26    | 25:21 | 20:16 | 15:0    |       |       |
| b. Load or store instruction |          |       |       |         |       |       |
| Field                        | 4        | rs    | rt    | address |       |       |
| Bit positions                | 31:26    | 25:21 | 20:16 | 15:0    |       |       |
| c. Branch instruction        |          |       |       |         |       |       |

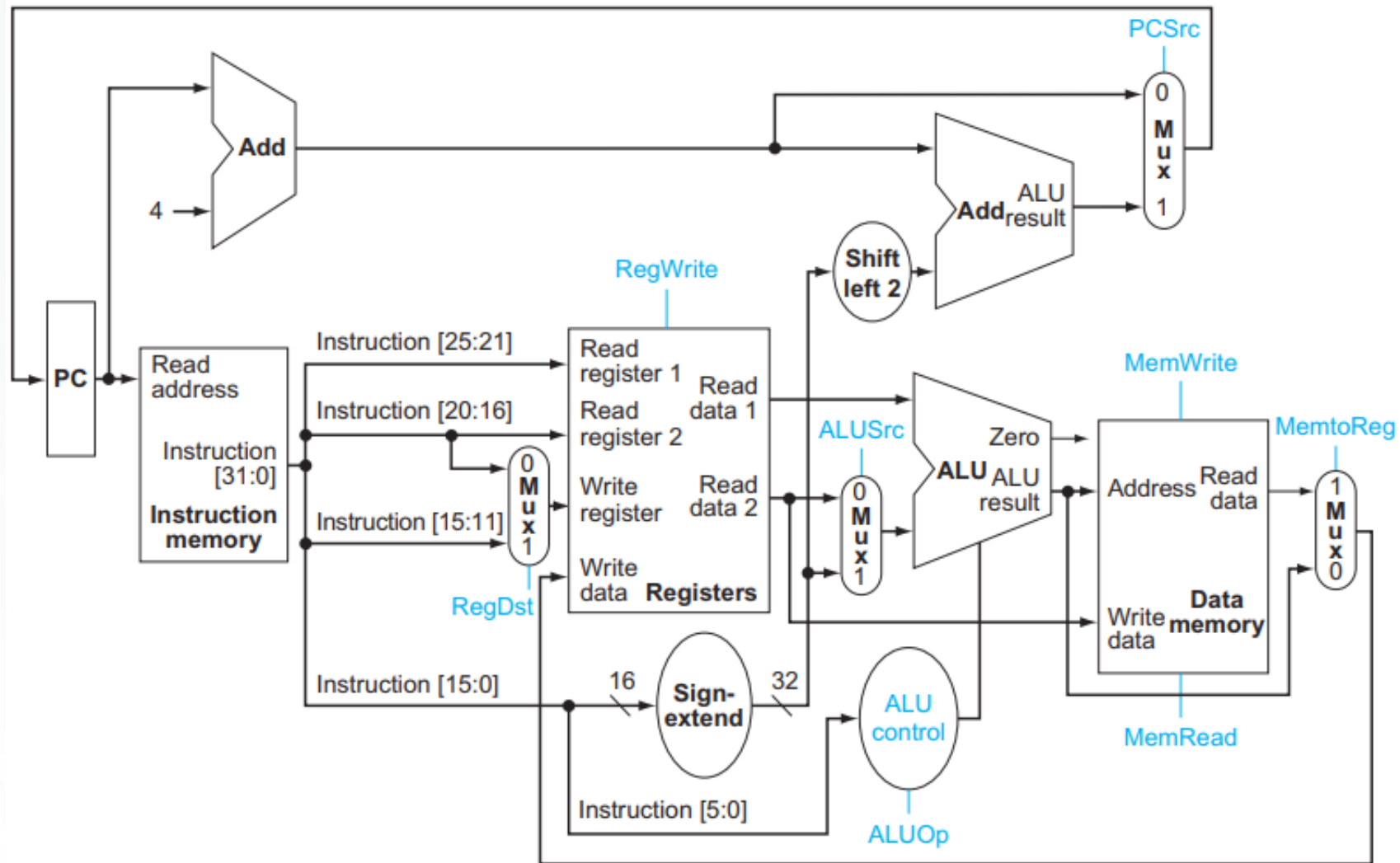
# ALU: PROJETO

- Aspectos relevantes sobre os campos das instruções
  - Registradores a serem **lidos**: `rs` e `rt`, campos 25:21 e 20:16 → R-type, beq, sw
  - Registradores **base** para `lw` e `sw`: `rs` campo 25:21
  - **16-bits offset** para Branch, Load e Store: campo 15:0
  - Registrador de destino :
    - (1) Load: registrador `rt` campo 20:16
    - (2) R-type: registrador `rd` campo 15:11

|                              |          |       |       |         |       |       |
|------------------------------|----------|-------|-------|---------|-------|-------|
| Field                        | 0        | rs    | rt    | rd      | shamt | funct |
| Bit positions                | 31:26    | 25:21 | 20:16 | 15:11   | 10:6  | 5:0   |
| a. R-type instruction        |          |       |       |         |       |       |
| Field                        | 35 or 43 | rs    | rt    | address |       |       |
| Bit positions                | 31:26    | 25:21 | 20:16 | 15:0    |       |       |
| b. Load or store instruction |          |       |       |         |       |       |
| Field                        | 4        | rs    | rt    | address |       |       |
| Bit positions                | 31:26    | 25:21 | 20:16 | 15:0    |       |       |
| c. Branch instruction        |          |       |       |         |       |       |

# ALU: PROJETO

- Caminho de dados (datapath) com Mux e sinais de controle



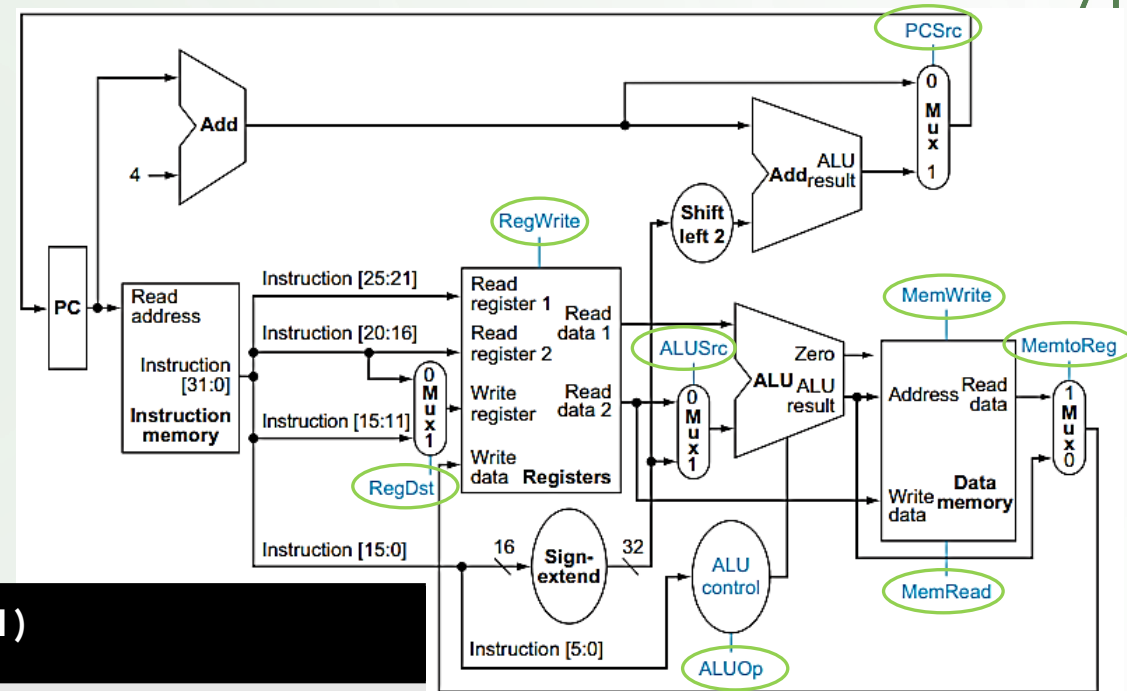
# ALU: PROJETO

• Devido ao novo caminho de dados (datapath) incluímos

- 1 sinal de 2 bits (**AluOp**)
- 7 sinais de controle de 1 bit

• Função dos 7 sinais de controle

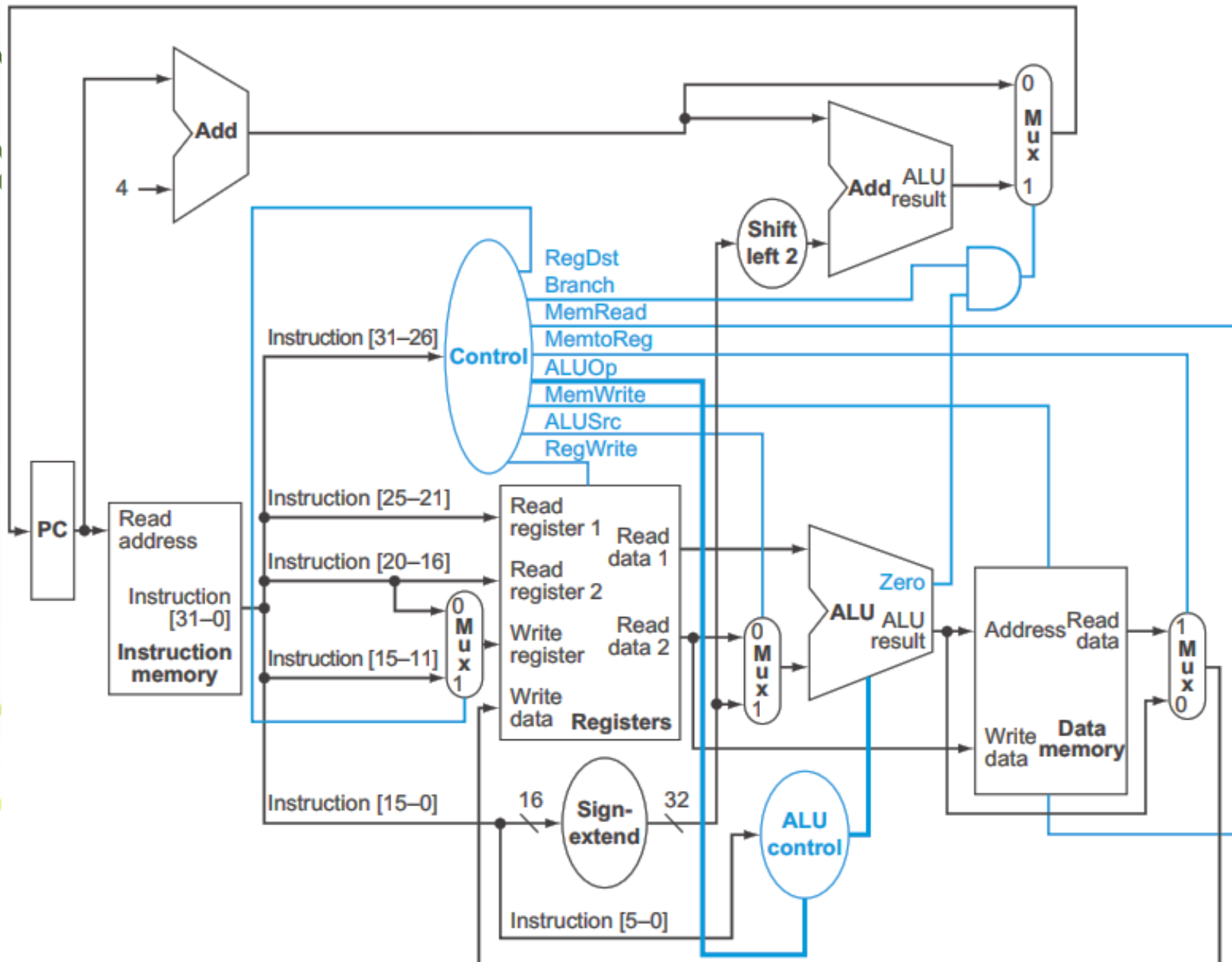
| Sinal           | Efeito quando inativo (0)                                    | Efeito quando ativo (1)                                                            |
|-----------------|--------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>RegDest</b>  | o nº do WriteRegister vem de InstructionMem 20:16, <i>rt</i> | o nº de WriteRegister vem de InstructionMem 15:11, <i>rd</i>                       |
| <b>RegWrite</b> | Nenhum                                                       | a entrada WriteData é escrita no registrador de nº WriteRegister                   |
| <b>ALUSrc</b>   | o 2º operando da ALU vem de ReadData2                        | o 2º operando da ALU vem de Sign-extend 16→32b vindo de InstructionMem offset 15:0 |
| <b>PCSrc</b>    | PC recebe o valor de PC+4                                    | PC recebe o valor de PC+4 + offset 15:0 de Sign-extend 16→32b                      |
| <b>MemRead</b>  | Nenhum                                                       | o conteúdo de DataMem-Address sai por DataMem-ReadData                             |
| <b>MemWrite</b> | Nenhum                                                       | o conteúdo de DataMem-Address chega por DataMem-WriteData                          |
| <b>MemToReg</b> | o valor de Register-WriteData vem da saída da ALU principal  | o valor de Register-WriteData vem da saída de DataMem-ReadData                     |



# ALU: PROJETO

- 6 dos 7 bits/sinais de controle podem ser definidos com base no Opcode
  - RegDest, RegWrite, ALUSrc, MemRed, MemWrite e MemToReg
- Os 2 sinais da unidade funcional ALU-control já foram especificados anteriormente (slide 29)
- O sinal PCSrc é definido com base em dois eventos
  - Se a instrução for um Branch
  - Se a saída Zero da ALU for True (1)
  - Assim, realizamos a operação AND entre o sinal de controle Branch e a saída ALU-Zero
- Todos os 7 sinais de controle de saída podem ser definidos por 6 sinais de entrada na Unidade de Controle, com base nos 6 bits da instrução (MSD ) 31:26

# ALU: SINAIS DA UNIDADE DE CONTROLE (CONTROL)



## Entrada:

- Campo Opcode da instrução 31:26

## Saídas:

- 3 sinais de 1 bit para as Mux
  - RegDst, ALUSrc e MemtoReg
- 3 sinais de 1 bit para R/W do Banco de registradores e DataMemory:
  - RegWrite, MemRead e MemWrite
- 1 sinal de 1 bit para Branch
  - Branch
- 1 sinal de 1 bit para a ALU
  - ALUOp



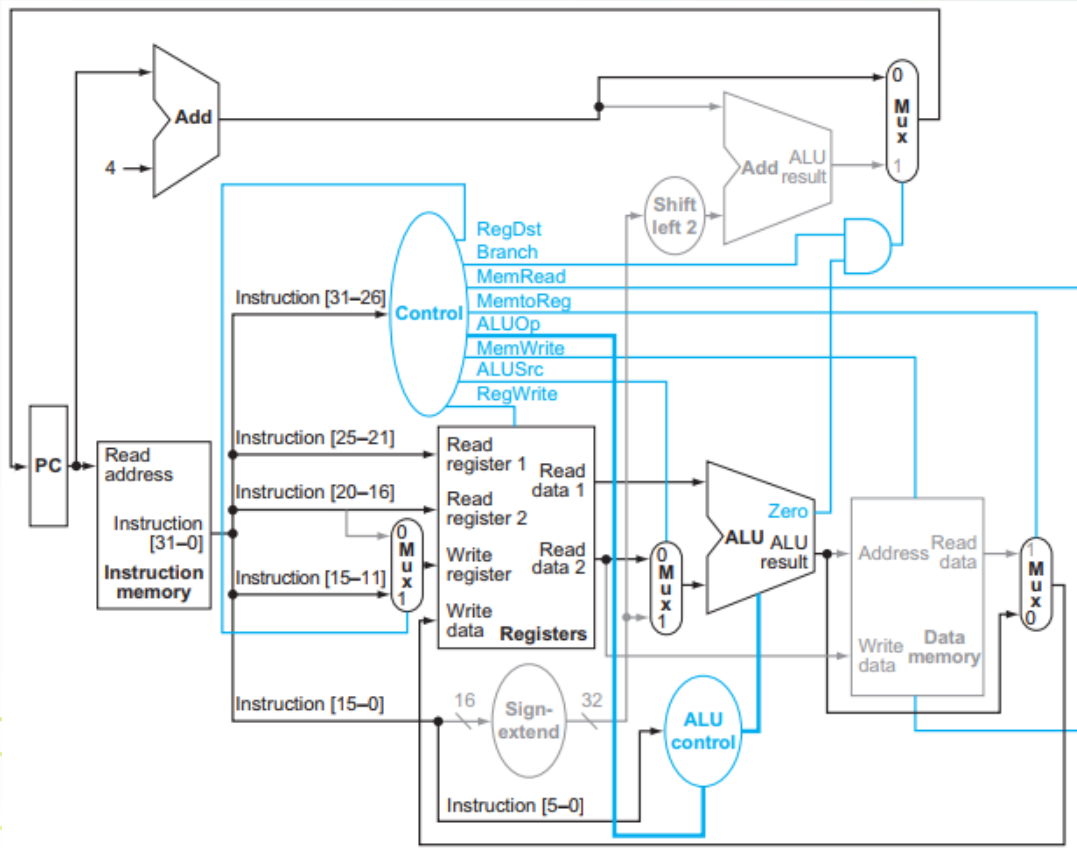
# ALU: DEFINIÇÃO DAS LINHAS DE CONTROLE A PARTIR DO OPCODE

| Instruction | RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | ALUOp1 | ALUOp0 |
|-------------|--------|--------|-----------|-----------|----------|-----------|--------|--------|--------|
| R-format    | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 1      | 0      |
| lw          | 0      | 1      | 1         | 1         | 1        | 0         | 0      | 0      | 0      |
| sw          | X      | 1      | X         | 0         | 0        | 1         | 0      | 0      | 0      |
| beq         | X      | 0      | X         | 0         | 0        | 0         | 1      | 0      | 1      |

- Linha 1, R-Type:
  - RegDest=1 seleciona RD, AluSrc=0 2º operando é registrador, MemToReg=0 saída ALU para registrador, RegWrite=1 escrita em registrador, MemRead/MemWrite/Branch=0 inativos, AluOP=10 aritmética
- Linha 2, Load:
  - RegDest=0 seleciona RT, AluSrc=1 2º operando é offset, MemToReg=1 saída Mem para registrador, RegWrite=1 escrita em registrador, MemRead=1 leitura Mem, MemWrite/Branch=0 inativos, AluOP=00 R/W
- Linha 3, Store:
  - RegDest=X don't care, AluSrc=1 2º operando é offset, MemToReg=X don't care, RegWrite/MemRead=0 inativos, MemWrite=1 escrita em Mem, Branch=0 inativo, AluOP=00 R/W
- Linha 4, Branch:
  - RegDest=X don't care, AluSrc=0 2º operando é registrador, MemToReg=X don't care, RegWrite/MemRead/MemWrite=0 inativos, Branch=1 usa offset, AluOP=01 branch

# ALU: OPERAÇÃO DO DATAPATH

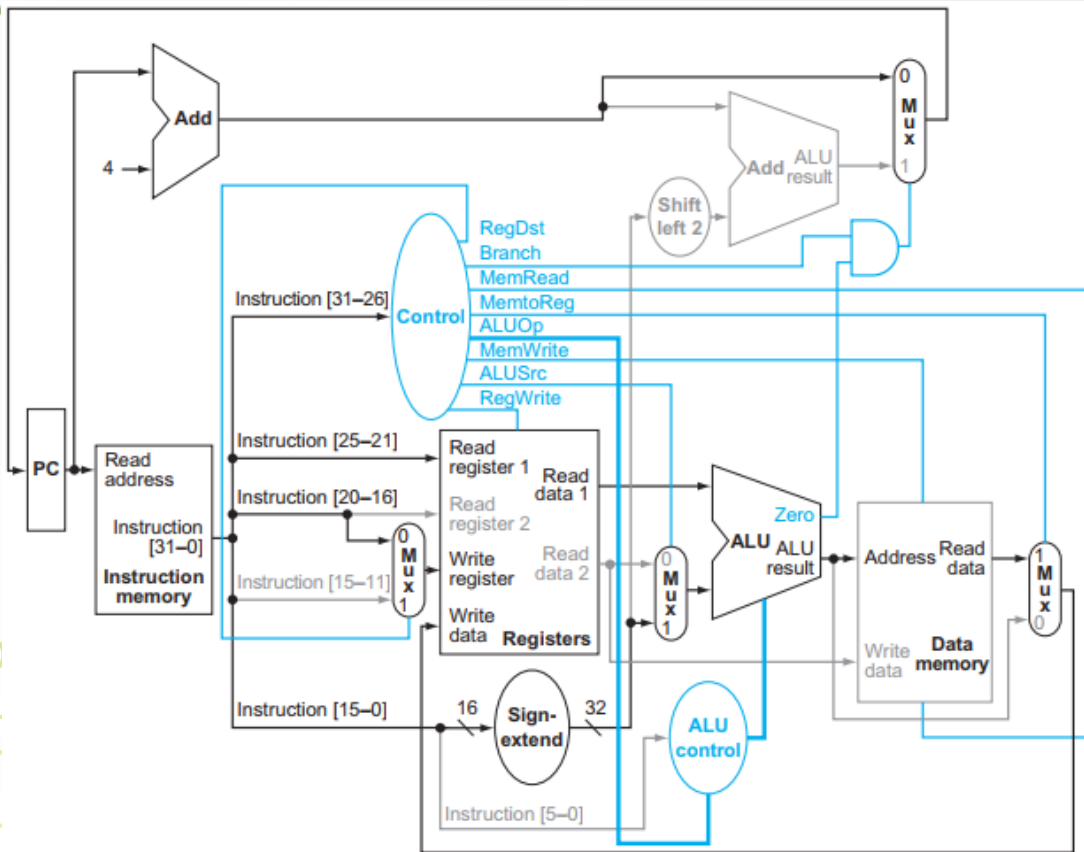
- Execução de instruções



- Tipo R, exemplo `add $t1, $t2, $t3`
- Etapas da execução:
  - **Busca** da instrução: da memória de instruções
  - **Decodifica**: com base no código de operação (OpCod 31:26) e função (Funct 5:0)
  - **Leitura de operandos**: 2 registradores do BR (Rs/Rt 25:16)
  - **Execução** na ALU: função definida pelo campo funct 5:0
  - **Resultado** da ALU: escrito no registrador destino, Rd 15:11

# ALU: OPERAÇÃO DO DATAPATH

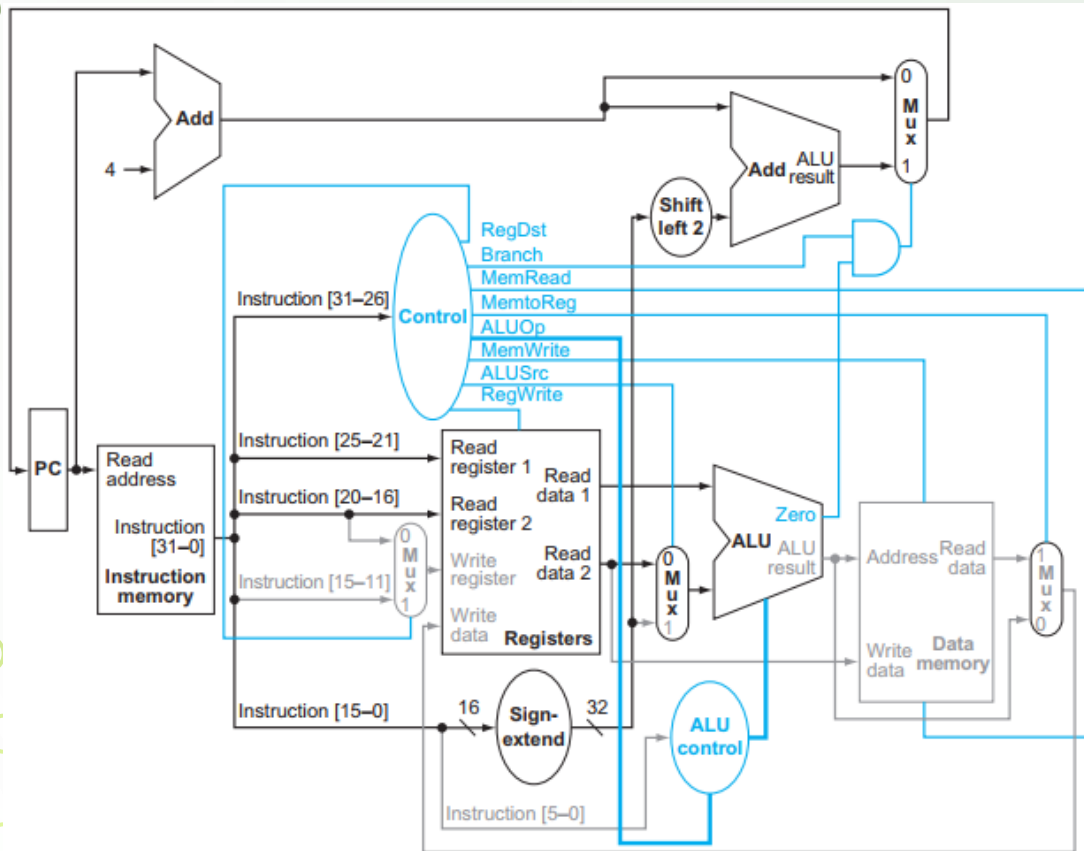
- Execução de instruções



- Tipo I, exemplo `lw $t1, offset($t2)`
- Etapas da execução:
  - **Busca** da instrução: da memória de instruções
  - **Decodifica**: com base no código de operação (OpCod 31:26)
  - **Leitura** do **operando**: 1 registrador do BR (Rs 25:21)
  - **Execução** na ALU: calcula o endereço de leitura, com base no registrador e o offset estendido 16-32b (Imm 15:0)
  - **Resultado** da ALU: conteúdo da memória de dados é escrito no registrador destino, campo Rt 20:16

# ALU: OPERAÇÃO DO DATAPATH

- Execução de instruções



- Tipo I, exemplo `beq $t1, $t2, offset`
- Etapas da execução:
  - **Busca** da instrução: da memória de instruções
  - **Decodifica**: com base no código de operação (OpCod 31:26)
  - **Leitura** do **operandos**: 2 registradores do BR (Rs/Rt 25:16)
  - **Execução** na ALU: calcula a diferença entre os registradores; soma PC+4 ao offset estendido 16-32b (Imm 15:0) e desloca 2 bits à esquerda
  - **Resultado** da ALU: bom base na saídas Zero da ALU seleciona que valor armazenar em PC: PC+4 ou resultado da soma/deslocamento

# ALU: CONTROLES

- Definidas as linhas de controle a partir do Opcode (slide 38) e

| Instruction | RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | ALUOp1 | ALUOp0 |
|-------------|--------|--------|-----------|-----------|----------|-----------|--------|--------|--------|
| R-format    | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 1      | 0      |
| lw          | 0      | 1      | 1         | 1         | 1        | 0         | 0      | 0      | 0      |
| sw          | X      | 1      | X         | 0         | 0        | 1         | 0      | 0      | 0      |
| beq         | X      | 0      | X         | 0         | 0        | 0         | 1      | 0      | 1      |

- Analisadas as etapas do caminho de dados (slides 39-41)
- Pode ser definida a função de controle (TV) para a UC

| Input or output | Signal name | R-format | lw | sw | beq |
|-----------------|-------------|----------|----|----|-----|
| Inputs          | Op5         | 0        | 1  | 1  | 0   |
|                 | Op4         | 0        | 0  | 0  | 0   |
|                 | Op3         | 0        | 0  | 1  | 0   |
|                 | Op2         | 0        | 0  | 0  | 1   |
|                 | Op1         | 0        | 1  | 1  | 0   |
|                 | Op0         | 0        | 1  | 1  | 0   |
| Outputs         | RegDst      | 1        | 0  | X  | X   |
|                 | ALUSrc      | 0        | 1  | 1  | 0   |
|                 | MemtoReg    | 0        | 1  | X  | X   |
|                 | RegWrite    | 1        | 1  | 0  | 0   |
|                 | MemRead     | 0        | 1  | 0  | 0   |
|                 | MemWrite    | 0        | 0  | 1  | 0   |
|                 | Branch      | 0        | 0  | 0  | 1   |
|                 | ALUOp1      | 1        | 0  | 0  | 0   |
|                 | ALUOp0      | 0        | 0  | 0  | 1   |

TV: Tabela Verdade  
UC: Unidade de Controle